DISEÑO DIGITAL 1.

TÍTULO DE LA ACTIVIDAD: **Ejercicios de modelado y simulación con varios procesos.**	**CÓDIGO:** **ANP6**

FECHA:			
NOMBRE:		APELLIDOS:	

MODALIDAD:	Libre. Ejercicios.	TIPO:	No Presencial	DURACIÓN:	120 minutos

CALENDARIO:	Antes de sesión presencial S9.	REQUISITOS:	Modelado en VHDL de circuitos secuenciales.

CRITERIO DE ÉXITO:	

COMENTARIOS E INCIDENCIAS:

TIEMPO DEDICADO:	minutos	AUTOEVALUACIÓN: [entre 0 y 10 puntos]	No procede

Introducción

En esta actividad verá un ejemplo de cómo pueden modelarse contadores complejos en VHDL empleando varios procesos. Verá también un ejemplo de cómo modelar un autómata de Mealy.

Ejercicio 1 – Contador BCD de dos dígitos

En la actividad anterior aprendió a modelar en VHDL un contador de módulo arbitrario, M, generando a su vez una salida de fin de cuenta. Por otro lado, en la actividad AR3 repasó conceptos sobre contadores adquiridos en la asignatura "Electrónica II", entre ellos cómo crear un contador BCD de varios dígitos mediante la conexión en cascada de varios contadores BCD de un dígito. En este ejercicio verá cómo modelar un contador de este tipo.

En la Figura 1 puede ver la estructura y el símbolo de un contador BCD de un dígito con entrada y salida de acarreo.

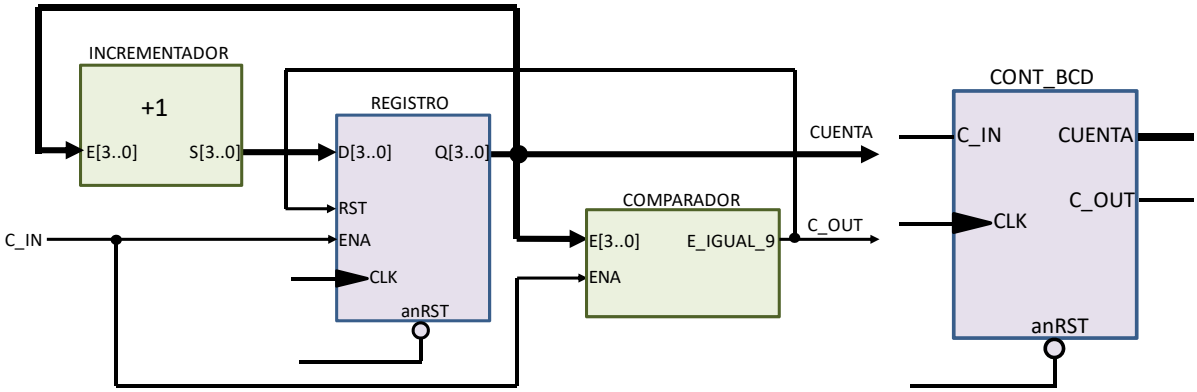


Figura 1: Contador BCD de un dígito con entrada y salida de acarreo (izquierda) y su símbolo (derecha)

Utilizando dicho contador como base la construcción de contadores BCD de varios dígitos es inmediata, tal como ya ha visto en ocasiones anteriores. En la Figura 2 puede ver por ejemplo cómo se construye un contador BCD de dos dígitos.

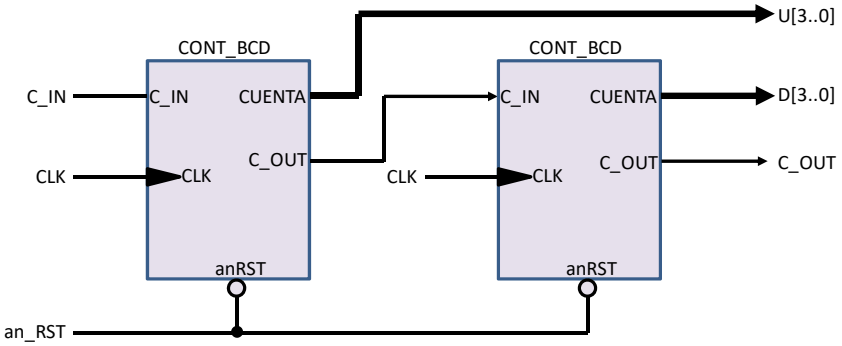


Figura 2: Contador BCD de dos dígitos realizado mediante contadores de un dígito

El modelado VHDL de un contador como el que se acaba de mostrar se puede realizar de modo análogo. En actividades anteriores ha modelado contadores con salida de fin de cuenta utilizando dos procesos, uno para modelar el conjunto de incrementador y registro y otro para modelar el circuito combinacional de generación de fin de cuenta.

Es posible extender la idea de emplear un proceso para modelar cada elemento de la estructura de un circuito determinado a circuitos más complejos, creando cuantos procesos sean necesarios dentro de un mismo cuerpo de arquitectura y declarando señales locales a éste para interconectar esos procesos entre sí. Considere el siguiente modelo del contador BCD de un dígito presentado en la Figura 1:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity cont_bcd is
port( clk, an_rst : in std_logic;
      c_in        : in std_logic;
      count       : buffer std_logic_vector(3 downto 0);
      c_out       : buffer std_logic
    );
end entity cont_bcd;

architecture rtl of cont_bcd is
begin

  proc_eoc: process(c_in, count)
  begin
    if c_in = '1' and count ="1001"then
      c_out <= '1';
    else
      c_out <= '0';
    end if;
  end process proc_eoc;

  proc_count: process(clk, an_rst)
  begin
    if an_rst = '0' then
      count <= (others => '0');
    elsif clk'event and clk = '1' then
      if c_out = '1' then
        count <= (others => '0');
      elsif c_in = '1' then
        count <= count +1;
      end if;
     end if;
  end process proc_count;

end architecture rtl;-- of cont_bcd
```

Estudie con atención el código del modelo. Fíjese en cómo el proceso `proc_count` modela el conjunto de incrementador y registro del esquema de la Figura 1, mientras que el proceso `proc_eoc` hace lo propio con el comparador que genera la señal de acarreo.

Compare ahora el código de dicho modelo con el que se presenta a continuación:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity cont_bcd_2dig is
port( clk, an_rst : in std_logic;
      c_in        : in std_logic;
      d, u        : buffer std_logic_vector(3 downto 0);
      c_out       : buffer std_logic
    );
end cont_bcd_2dig;
```

```
architecture rtl of cont_bcd_2dig is
  signal c_u : std_logic;
begin

  proc_count_u: process(clk, an_rst)
  begin
    if an_rst = '0' then
      u <= (others => '0');
    elsif clk'event and clk = '1' then
      if c_u = '1' then
        u <= (others => '0');
      elsif c_in = '1' then
        u <= u +1;
      end if;
    end if;
  end process proc_count_u;

  proc_c_u: process(c_in, u)
  begin
    if c_in = '1' and u = "1001" then
      c_u <= '1';
    else
      c_u <= '0';
    endif;
  end process proc_c_u;

  proc_count_d: process(clk, an_rst)
  begin
    if an_rst = '0' then
      d <= (others => '0');
    elsif clk'event and clk = '1' then
      if c_out = '1' then
        d <=(others=> '0');
      elsif c_u = '1' then
        d <= d +1;
      end if;
    end if;
  end process proc_count_d;

  proc_c_out: process(c_u, d)
  begin
    if c_u = '1' and d = "1001" then
      c_out <= '1';
    else
      c_out <= '0';
    end if;
  end process proc_c_out;

end architecture rtl;-- of cont_bcd_2dig
```

Observe que el cuerpo de arquitectura contiene dos parejas de procesos (`proc_count_u`/`proc_carry_u` y `proc_count_d`/`proc_c_out`) análogas a la presente en el modelo del contador BCD de un dígito. En el cuerpo de arquitectura se declara una señal interna llamada `c_u`, de tipo `std_logic`, mediante la cual se conecta la salida de acarreo del contador de unidades a la entrada de acarreo del contador de decenas.

Realice las siguientes operaciones:

1. Descargue de Moodle los ficheros correspondientes a esta actividad.

2. Suponga que en un instante dado el contador modelado por el código anterior se encuentra en el estado `d = “1000”`, `u = “1001”`. Suponga también que la señal `clk = ‘0’` y que `c_in = ‘1’`.

Deduzca a partir del código del modelo cuál será la secuencia de activación de cada uno de los procesos en el siguiente flanco de subida de la señal `clk`, y escriba en el anexo 1 dicha secuencia y las asignaciones que se realizan en cada activación, utilizando el formato que en el mismo se indica.

3. Cree un proyecto ModelSim con el nombre "Proy_ANP6" y añada al mismo el fichero que contiene el modelo del contador BCD de dos dígitos que acaba de analizar y su banco de test, que encontrará en la ubicación habitual con los nombres *cont_bcd_2dig.vhd* y *tb_cont_bcd_2dig.vhd*, respectivamente.

4. Inicie la simulación y añada al visor de formas de onda todas las entradas y salidas del contador *cont_bcd_2dig*. Simule el banco de test hasta el instante t = 1835 ns, escribiendo en la ventana *Transcript* el comando `run 1835ns`.

5. Ejecute el comando `step` en la ventana *Transcript*. Verá cómo se abre el fichero cont_bcd_2dig.vhd en el editor (si no lo tiene ya abierto) y aparece una flecha azul señalando la sentencia wait de la línea 40. Verá también cómo en la ventana *Processes (Active)* aparece el proceso `proc_clk`; si no puede ver esta ventana, actívela mediante la opción *Process* del menú *View*. Fíjese además en que en la parte inferior de la ventana de ModelSim, en la barra de información, puede ver el tiempo actual de simulación (1840 ns) y el número de ciclo delta que se está ejecutando para ese instante (0, en este caso).

   El comando `step` permite ejecutar la simulación paso a paso, lo que le permitirá ver qué procesos se activan en cada ciclo delta de simulación (son los que aparecen en la ventana *Processes*) y qué sentencias se ejecutan en la activación de cada proceso.

6. Continúe la ejecución de la simulación mediante comandos `step` (puede usar también el botón de la barra de herramientas de ModelSim) hasta que el tiempo de simulación llegue a 1850 ns, y utilice la información que obtenga de la ejecución paso a paso para rellenar en el anexo 2 la secuencia de activación de los procesos y las asignaciones realizadas en cada activación, como hizo anteriormente.

7. Compare la información que ha escrito en los anexos I y II: debería ser esencialmente la misma. En el anexo III encontrará la solución. Debe comprobar cualquier discrepancia con lo que ha escrito en los anexos I y II.

Ejercicio 2 – Conformador de pulsos

Un conformador de pulsos es un circuito secuencial que genera un pulso de duración constante (habitualmente un ciclo de reloj) en su salida cuando recibe en su entrada un pulso de duración arbitraria. El cronograma de la Figura 3 muestra un ejemplo de funcionamiento en el que *e* es la señal de entrada y *s* la de salida.

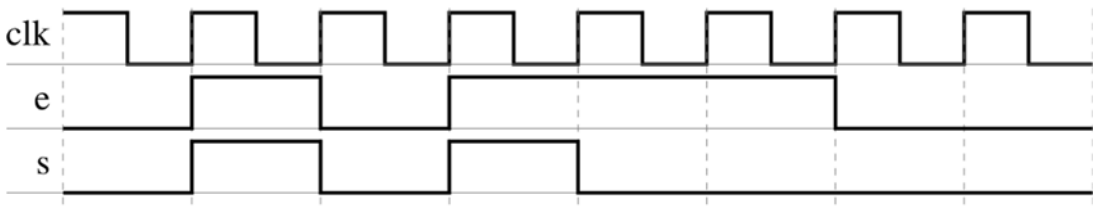


Figura 3: Ejemplo de funcionamiento de un conformador de pulsos

Este conformador de pulsos puede verse como un autómata que necesariamente debe responder a la arquitectura de Mealy, ya que, como se ve en el ejemplo mostrado, la salida reacciona a un cambio en la entrada en el mismo ciclo de reloj en el que este se produce. El esquema del circuito puede verse en la Figura 4[1].

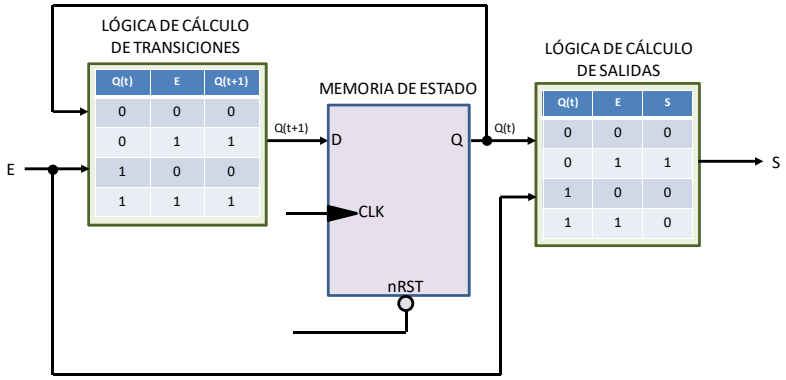


Figura 4: Estructura del conformador de pulsos

Realice las siguientes operaciones:

1. Añada el fichero *conformador.vhd*, que encontrará en el lugar habitual, al proyecto ModelSim que está utilizando.

2. En dicho fichero encontrará un modelo incompleto del conformador de pulsos. Complételo de modo que ante un pulso en la entrada de cualquier longitud presente en su salida un pulso de un ciclo de reloj de duración.

3. Codifique un banco de test para el conformador. Guárdelo en el fichero *tb_conformador.vhd* y añádalo al proyecto.

4. Simule el conformador y verifique que su funcionamiento se corresponde con la especificación.

[1] Si observa con atención la lógica correspondiente al cálculo de transiciones verá que, en realidad, la salida correspondiente al estado futuro coincide siempre con el valor de la entrada E de manera que, en este caso, la "lógica" es simplemente un "cable" (conexión directa de E a la entrada del flip-flop de la memoria de estado).

Anexo I: Análisis manual de la secuencia de activación de procesos

Cada fila en la siguiente tabla representa un ciclo delta. Escriba en la columna de la izquierda las etiquetas de los procesos que se activan en cada ciclo delta, y en la columna derecha las asignaciones que se producen en cada activación, como se muestra en el siguiente ejemplo:

proc_count_u proc_count_d	u <= “0110” d <= “0010”

Rellene a continuación la información que se le pide.

Anexo II: Análisis con ModelSim de la secuencia de activación de procesos

Cada fila en la siguiente tabla representa un ciclo delta. Escriba en la columna de la izquierda las etiquetas de los procesos que se activan en cada ciclo delta, y en la columna derecha las asignaciones que se producen en cada activación, utilizando el mismo formato que en el Anexo I.

Anexo III: Solución

Tabla del Anexo I:

proc_count_u proc_count_d	u <= “0000” d <= “1001”
proc_carry_u proc_carry_d	c_u<= ‘0’ c_out<= ‘1’
proc_carry_d	c_out<= ‘0’

Tabla del Anexo II:

proc_clk	clk<= ‘1’
proc_count_u proc_count_d	u <= “0000” d <= “1001”
proc_carry_u proc_carry_d	c_u<= ‘0’ c_out<= ‘1’
proc_carry_d	c_out<= ‘0’
proc_clk	clk<= ‘0’